Final Year Major Project - CSE

SMART CAMPUS AGENT SYSTEM

Viva & Interview Notes

everything to say, and every question back

Name Sujan

Date 05 / 09 / 2026

What's in here

1. The 60-second pitch -> say this first
2. Why the project exists
3. Stack, and why each piece
4. How one request actually flows
5. Six decisions worth defending
6. Numbers + the 12 hard constraints
7. Cross-questions with answers (6 groups)
8. The hard ones - answer honestly
9. The 5-minute demo path

Read from the source, not the summary.

1. THE 60-SECOND PITCH

Say this first, in this order: problem -> what it does -> the ONE architectural idea -> proof it works.

> College ops run on 2 jobs nobody enjoys: building the weekly timetable, and patching it every time a teacher takes leave. We built a multi-agent system that does both.
>
> An admin types one sentence - "add a teacher John who takes DBMS for CSE-7A" - and an LLM turns it into a validated, TYPED plan, previews it in English, applies it. Then an OR-Tools CP-SAT solver builds a clash-free timetable from the rulebook.
>
> When a leave is approved, nobody prompts anything: a LangGraph agent fires on the event, builds a PERIOD-EXCHANGE plan, then pauses itself with interrupt() until the HOD clicks approve - and resumes from exactly that line, even after a server restart.

The core idea is division of labour:

The LLM NEVER schedules anything.

It translates intent into typed operations. A deterministic solver does the maths. A human approves anything that touches real people.

2. WHY IT EXISTS

Timetabling is a multi-week manual job done in spreadsheets, and it breaks the moment reality intrudes. Two failure modes we targeted:

- Generation is slow + OPAQUE. When a timetable can't be built, a spreadsheet gives no reason. Our solver returns the named conflicting constraint groups + concrete fixes.
- Substitution DESTROYS the timetable. The usual 'find any free teacher to cover' puts the wrong person in front of the class. We model what colleges actually do -> a period exchange.

The gap we position against

- Smart-campus research = reactive (IoT sensing, dashboards).
- Timetabling research = genetic algorithms, which cannot explain infeasibility.
- Faculty-substitution work = stops at classification.

→ Nobody combines proactive multi-agent orchestration + a constraint solver + human-in-the-loop approval for campus ops.

3. THE STACK, AND WHY EACH PIECE

They will ask 'why this and not X' for at least two of these. The right-hand side IS the answer.

LangGraph	needs a durable, RESUMABLE graph. interrupt() + a checkpointer is what makes days-long human approval possible. A plain chain cannot pause.
OR-Tools CP-SAT	timetabling is a constraint-satisfaction problem, not a text problem. Exact, fast, and can PROVE infeasibility + name the cause.
LLM layer	one factory, init_chat_model(). Gemini / Claude / OpenAI / Ollama is an .env change. No module imports a provider SDK.
FastAPI	Pydantic validation shared with the agent's structured output; auto OpenAPI; async-ready.
SQLAlchemy 2	20 tables, no raw SQL anywhere. SQLite for dev - the DSN is config, so Postgres is a URL change.
APScheduler	in-process 2-min sweep as a safety net for the event path. Enough at one-campus scale; Celery if it grows.
PyJWT + passlib	HS256 bearer tokens, pbkdf2_sha256 hashing, a require_role() dependency factory on every write route.
Next.js	10 pages, role-aware. PDF export is client-side jsPDF, so no server-render dependency.

4. HOW ONE REQUEST ACTUALLY FLOWS

Two entry points converge on the SAME graph: a chat message, or a system event.

Path A - an admin asks for a change

- 1. Admin types one sentence on /constraints
- 2. LLM emits typed ops as JSON - never SQL, never prose
- 3. Each op validated against live DB entities, in order
- 4. Whole plan rendered in ENGLISH. Nothing written yet.
- 5. One confirmation -> one transaction, all or nothing
- 6. CP-SAT re-plans, preserving the published week

Path B - a leave gets approved (nobody prompts anything)

- 1. Leave approved -> API invokes the graph with source='system'
- 2. Supervisor short-circuits to 'substitution' - NO LLM call
- 3. Period exchanges built for every disrupted lesson
- 4. interrupt() freezes the node; thread id saved on Approval row
- 5. HOD approves -> Command(resume=...) restarts that exact node
- 6. Exchanges confirmed; partner + returning teacher get notified

5. SIX DECISIONS WORTH DEFENDING

If you memorise ONE section, make it this one. Each has a sentence you can say word for word.

1. **The LLM orchestrates; it never schedules**

   The model's job is upstream (turn a sentence into typed constraint rows + operations) and downstream (explain an outcome). Placement of 48+ lessons is done by CP-SAT, deterministically.

   > "An LLM that guesses a timetable gives you a plausible-looking one with clashes in it. We made the LLM a translator and gave the maths to a solver that can prove its answer."

2. **The constraint registry is the single source of truth**

   POST /api/timetable/generate takes NO request body. It compiles the constraints table into solver options. So whatever an admin can see and edit on the Constraints page is exactly what produced the timetable - no hidden state, and rules stay editable after the AI writes them.

   The catalog (14 rule kinds) declares scopes, form fields, a validator and an English renderer per kind - and BOTH the UI form and the LLM prompt are generated from it. Adding a kind touches one registration + the solver.

   > "The AI writes rules into the same table a human edits by hand. It's a fast path to the rulebook, not a parallel one."

3. **Natural language -> 23 typed operations, with forward refs**

   The model returns {'ops':[{'op':'add_teacher','params':{...}}]}. Each op has validate / describe / apply. A plan is walked in dependency order carrying a PlanCtx.

   Ops that create entities register them as PENDING with negative ids, so a later op can reference a class the same plan is creating - then everything is re-validated for real at apply time.

   Bulk forms exist for a real reason: OUTPUT TOKENS are what the admin waits on. A 5-subject request went from 18 longhand ops that timed out, to one compact op.

> "The model can't write SQL and can't write free text - it can only fill in operations we defined, and every one is validated against live data before anything is written."

4 Period exchange, NOT substitute cover

Instead of 'find a free teacher to babysit the class', a partner teacher of the SAME section moves their own lesson into the leave slot, and the absent teacher recovers the missed lesson later in the partner's vacated slot.

- No subject is ever taught by an unqualified person.
- Weekly period counts preserved - lessons just swap dates.
- Original timetable_entries are NEVER mutated. Each exchange is a dated overlay row in period_exchanges, grouped under one plan_id, so the HOD approves/rejects atomically.
- /api/timetable/effective/{section}?date= composes the real day on demand.

The planner PENALISES hectic outcomes rather than banning them: a swap creating same-subject back-to-back for students, or a 3rd consecutive teaching period, gets a warning on the approval card and loses to any clean swap - but still beats 'no exchange at all'.

> "We modelled what colleges actually do, which is why the timetable survives the leave instead of being rewritten by it."

5 Human-in-the-loop that survives a restart

The substitution node calls interrupt(). LangGraph's SqliteSaver checkpoints the thread; the Approval row stores langgraph_thread_id. POST /api/approvals/{id}/decide calls invoke(Command(resume={'action':'approve'}), thread_id=...) and the node continues from the interrupt - days later if need be.

!! SAY THIS UNPROMPTED: on resume, LangGraph RE-EXECUTES the node from the top. So everything before the interrupt must be idempotent. build_plan() returns the existing plan if one exists; the Approval insert is an upsert.

> "Autonomy without a brake is a liability. The agent proposes; a human commits - and the pause is durable, not an in-memory promise."

6 Regeneration that leaves the published timetable alone

CP-SAT has no memory of its last answer - re-solving after one small edit is free to return a completely different, equally optimal week, which is useless to a college that already published one.

So the previous placement is fed back TWO ways: as AddHint() on every variable, AND as a penalty term sum(1 - var) in the objective.

Hints alone only bias the search. The penalty is what makes "don't change what you don't have to" an actual objective.

Result: adding a subject moves ZERO existing lessons and solves in ~0.6s, vs ~8s and a rewritten week for a cold ?fresh=true re-plan.

> "Minimal-change re-planning is the difference between a demo and something a college would actually run."

6. NUMBERS TO HAVE READY

Quote these. They make the work sound measured rather than assembled.

20	DB tables (SQLAlchemy). No raw SQL in agents or tools.
H1 - H12	hard constraints, + a weighted soft objective
14	rule kinds in the catalog (scopes, validator, renderer each)
23	assistant operations - typed, validated, previewed, atomic
~7,300	lines of backend Python. Solver 985, ops 1112, constraints 774.
~4,700	lines of frontend TSX across 10 role-aware pages
~8 s	cold solve. 48 lessons, ZERO clashes verified by independent SQL.
~0.6 s	preserving solve. Adding a subject moves 0 lessons.
20s / 8	solver budget: max_time_in_seconds / num_workers
2 min	APScheduler safety sweep. coalesce, max_instances=1.

The 12 hard constraints - know these cold

H1	every (section, subject) gets exactly periods_per_week lessons
H2	a section has at most one lesson per timeslot
H3	a teacher teaches at most 1 lesson per timeslot - counting open-elective duty
H4	exactly one QUALIFIED teacher per (section, subject), pinned when the admin assigned one
H5	lab subjects run as one consecutive same-day block in a lab room; lab rooms clash-free
H6	each section gets a dedicated home classroom with enough capacity
H7	teacher daily load <= max_hours_per_day (weekly cap optional)
H8	at most subject_daily_max periods of the same subject per day
H9	no same-subject back-to-back for theory (scoped, ON by default)
H10	optional cap on a teacher's consecutive teaching periods
H11	open-elective bands: every section of the semester idle while baskets run in parallel
H12	availability bans - a teacher / subject / section forbidden at given slots

SOFT — fairness (busiest - idlest teacher load gap) + preferred-period misses + mid-day gaps + the move penalty

7. CROSS-QUESTIONS, WITH ANSWERS

Grouped the way panels actually move: architecture, then the solver, then the uncomfortable ones.

A. Architecture & agents

Q. Why multi-agent? Couldn't one LLM with tools do all of this?

A. It could, badly. Separate nodes give each domain its own prompt, toolset and failure boundary - a bad substitution plan can't corrupt timetable generation. More importantly LangGraph gives per-node checkpointing, so one node can pause for human approval while state stays durable. A single agent loop has no natural pause point and no per-domain trace - and the trace in our UI IS the list of node steps.

Q. Walk me through your graph. Nodes and edges?

A. A StateGraph(AgentState), entry point 'supervisor', then add_conditional_edges routing on route_to(state) to one of five specialists - timetable, substitution, facility, scheduler, general_fallback - each edging to END. Compiled with a SqliteSaver checkpointer. Supervisor-router topology, deliberately shallow: one hop, because every specialist is terminal.

Q. How does the supervisor route? Just keywords?

A. No - structured output. A Pydantic RouteDecision with a Literal route field + entity slots (room, date, time_range, subject) + a reasoning string, then get_llm().with_structured_output(RouteDecision). The Literal's docstring IS the routing instruction, so adding a route is one enum value. A deterministic keyword fallback sits underneath for when the provider is down or no key is configured.

Q. So there are two routers. Isn't that duplicated logic?

A. It's a deliberate degradation path, not duplicated intent - the fallback is coarse keyword matching whose only job is to keep the system usable with no LLM. ~12 lines, and it's what lets the whole thing run on a machine with no API key. Every node logs which path it took into steps, so it's visible in the trace, never silent.

Q. What's in your state? Why a TypedDict and not Pydantic?

A. LangGraph merges PARTIAL state updates from each node, and a TypedDict with Annotated reducers is its native shape. Our one reducer is steps: Annotated[List[str], operator.add] - step lists concatenate across nodes so the

trace accumulates. Every other field is last-write-wins. Fields: messages, current_action, steps, params, final_response, source, task_spec.

Q. Why does a system event skip the LLM?

A. Because the intent is already known. A leave approval invokes the graph with source='system' and task_spec={'leave_id': N}. The supervisor sees both and routes deterministically. Asking an LLM to classify a message we ourselves wrote adds a round-trip and a nonzero chance of misrouting, for no information gain. Both the approval endpoint and the sweep use this path.

Q. Why do agents never touch the database directly?

A. Auditability and safety. Every capability is a typed Python function in app/tools/, so every action is a logged tool call the trace sidebar can render. And permissions are enforced in tools and FastAPI dependencies, where they are CODE - an LLM cannot be trusted to enforce a permission it can also be talked out of.

Q. How would you add a new agent?

A. Write the node function in app/agents/specialists/, register it in graph.py with add_node + an edge to END + a branch in the conditional map, and add the route value to the supervisor's Literal. No keyword table to update - the routing description travels with the enum.

Q. Where does the trace in the UI come from?

A. The steps list. Each node appends human-readable lines - which provider routed, how many lessons were affected, which approval id it's waiting on. Cheap observability that doubles as the demo narrative.

B. The LLM layer & reliability

Q. What happens when the LLM hallucinates an operation?

A. It can't produce anything but a key from OPS, and unknown keys are either aliased or rejected before validation. Then each op's validate runs against live DB entities - a subject code that doesn't exist, a semester out of range, a capacity over cap all fail with a specific message. Then the whole plan is rendered in English for the admin before a single write. THREE GATES: the schema, the validator, the human.

Q. What if it emits create_teacher instead of add_teacher?

A. There's an alias map for exactly that drift - create_teacher, add_section,

add_constraint and ~15 others normalise to the canonical op. Rather than fighting model naming variance with prompt scolding, we absorb the common variants.

Q. How is it provider-agnostic in practice, not just in claim?

A. One rule enforced across the codebase: NO module imports a provider SDK. Everything calls get_llm(), which wraps init_chat_model(model, model_provider, temperature) and is lru_cached per temperature. is_llm_configured() resolves whether the provider has its key so callers degrade instead of crashing. Gemini -> Claude is two lines in .env.

Q. What's text_of() for? That looks like a workaround.

A. It's a real portability bug we hit. Older providers return content as a string; newer ones - Gemini, Claude with extended thinking - return a LIST of typed content blocks. Anything that str()s raw content gets a Python repr of a block list, which then fails JSON parsing confusingly. text_of() normalises both shapes, and every caller that parses JSON goes through it.

Q. How do you keep prompts from drifting out of sync with the code?

A. The rule catalog generates its own prompt section - the LLM is shown the 14 kinds with scopes and fields straight from CATALOG, and the same structure generates the UI form. Register a kind and both update. The op reference is currently a hand-maintained string in assistant.py - that one I'd generate from OPS too, and it's a fair thing to be asked about.

Q. Cost and latency of an LLM call?

A. One structured-output call per chat message for routing, one per assistant request for planning. NOTHING in the solve path calls an LLM at all. The assistant call is the slow one - up to 90s of budget, run in a worker thread - which is why the bulk op forms exist.

Q. Why temperature 0.1 for routing and 0.4 for general chat?

A. Routing is classification, where variance is pure downside. General campus Q&A benefits from slightly more natural phrasing. The factory caches per temperature so both share one client pool.

C. The solver - expect the deepest probing here

Q. Why CP-SAT and not a genetic algorithm?

A. Three reasons. It's EXACT - it returns optimal, not 'good enough after N generations'. It's fast at our scale. And it can PROVE infeasibility and say which constraint groups conflict, which a GA structurally cannot: a GA that fails just returns a bad chromosome. That last point is our main differentiation against the timetabling literature, where GAs dominate.

Q. What are your decision variables?

A. Booleans throughout. x[(section, subject, slot)] - is this lesson placed here. y[(section, subject, teacher)] - who teaches this pair, with AddExactlyOne over eligible teachers. c[(section, room)] - home classroom. lr[(section, subject, room)] - lab room. gv[(group, slot)] - elective band placement. Keeping everything boolean lets CP-SAT's presolve do most of the work.

Q. How exactly do you explain infeasibility?

A. Two layers. (1) Cheap PRECHECKS before the model is even built - 'Section CSE-7A needs 30 periods/week but its rules leave only 28 slots' - which catch most real cases with a precise human sentence.

A. (2) If still infeasible: CP-SAT ASSUMPTION LITERALS. One Boolean per constraint group, registered with AddAssumptions, every constraint in that group attached via OnlyEnforceIf. On infeasible we call SufficientAssumptionsForInfeasibility() and map the returned literal indices back to names - 'Conflicting constraint groups: teacher daily hour limits, pinned teacher assignments' - plus typical fixes.

Q. What's your objective function?

A. A weighted sum we minimise: the fairness gap (busiest minus idlest teacher's weekly load), preferred-period misses, mid-day gap periods for students, and - when preserving - the move penalty. Weights configurable per rule. It's single-objective by weighted scalarisation, not lexicographic, which is simpler to tune - and I'd say that rather than pretend it's Pareto-optimal.

Q. How do you model a mid-day gap? That's not obvious.

A. Per section per day: an occupancy Boolean per slot, then two running-max chains built with AddMaxEquality - before[k] = a class happens at or before k, after[k] =

at or after k. A gap at k is then gap >= before[k] + after[k] - occupied[k] - 1, i.e. idle, but the day has already started and hasn't ended. Sum those into the objective. Standard trick for interior-hole detection with linear constraints.

Q. How do lab blocks work?

A. A lab subject must occupy consecutive same-day periods in a lab room, and lab rooms are clash-free across sections. Adjacency is defined by TIME, not period number - two periods are adjacent only when prev.end == next.start, so a lunch break correctly BREAKS a run. The same helper defines adjacency for H9 and for the exchange planner's warnings.

Q. Explain scope resolution.

A. Every scoped rule resolves SECTION > SEMESTER > GLOBAL. A rule row carries scope_type + scope_value; at compile time we build per-semester and per-section override objects where every unset field is None, meaning 'the next scope up decides'. That's what lets ONE solve produce a genuinely differently-shaped week per semester - sem 6 with an elective band on Wednesday, sem 7 with a half day, from the same run.

Q. How do open electives work? That sounds hard.

A. It's the hardest constraint in the model. A band reserves the same periods across EVERY section of that semester - so all those sections are idle for regular lessons - while its baskets run in parallel, each in its own reserved room with its own teacher. Those rooms leave the home-classroom pool entirely so no section can own them, and every basket teacher is marked busy in those slots for the clash constraint. A band can optionally be pinned to fixed slots, with remaining band periods still floating.

Q. Is your timetable actually clash-free, or do you just trust the solver?

A. We verified INDEPENDENTLY - SQL queries over the stored entries checking no teacher, section or room appears twice in a timeslot, across three sections and multiple semesters. 13/13 checks passed on the Phase 2.4 run. Trusting the solver's own model would be circular; that's why the check reads the persisted rows.

Q. What if the solver times out?

A. 20-second budget, 8 workers. CP-SAT returns FEASIBLE rather than OPTIMAL if it found a solution but couldn't prove optimality - we accept both and

store the status. On UNKNOWN we return 422 with the status name rather than persist a partial timetable. Practically the preserving path solves in under a second, so the timeout is a backstop.

Q. How does it scale? What's the complexity?

A. Variable count is roughly sections x subjects x slots, so linear in each and it grows fast with departments. At our scale - a few sections x ~8 subjects x 35 slots - that's a few thousand Booleans, trivially fast. A whole university would need DECOMPOSITION: solve per department or per semester, since sections in different departments share only rooms and a few teachers. I'd partition on that sharing graph rather than throw a bigger time budget at it.

Q. Why hints AND a penalty for preservation? Isn't one enough?

A. No, and this is the subtle bit. AddHint() only BIASES the search - CP-SAT is free to abandon a hint the moment it finds an equally good solution elsewhere, and it usually does. The penalty term sum(1 - var) over the previous placement makes stability part of what's being minimised, so a move has to pay for itself. Hints buy speed; the penalty buys the guarantee.

D. Human-in-the-loop & LangGraph internals

Q. What does interrupt() actually do?

A. It raises a special exception LangGraph catches, checkpoints the graph state under the thread id, and returns control to the caller with the interrupt payload. The graph isn't running or blocked - it's PERSISTED. Resuming means invoking with Command(resume=value) and the same thread id; the interrupt call then returns that value and execution continues.

Q. You said the node re-executes on resume. Isn't that a bug?

A. It's the documented semantics - LangGraph REPLAYS the node up to the interrupt rather than snapshotting a Python stack frame. So it's a correctness requirement on us: everything before interrupt() must be idempotent. build_plan() returns the existing plan if plan_id already has rows, and the Approval lookup is get-or-create. I'd rather call this out than have it found.

Q. What if the server restarts while a plan awaits approval?

A. Nothing is lost. The SqliteSaver has the thread on disk and the Approval row has the thread id, so the approve click still resumes the right node. Separately an APScheduler sweep runs every 2 minutes and re-triggers the agent for any approved leave with no plan rows at all - the belt to the checkpointer's braces, for the case where the process died before the plan was even built.

Q. Why a scheduler sweep if the event path already works?

A. Because the event path is an inline call inside the approval request. If that request fails halfway, or the process is killed, the leave is approved with no plan and nobody would ever know. The sweep is a RECONCILIATION LOOP: desired state (approved leaves) vs actual state (existing plans), and it closes the gap. coalesce=True, max_instances=1, so it can't stack up.

Q. Can two admins approve the same plan at once?

A. The endpoint checks status != 'pending' and returns 409, so the second click is rejected. Honestly that's a check-then-act against SQLite rather than a row lock, so there's a narrow race window. On Postgres I'd make it a conditional update - UPDATE ... WHERE status='pending' - and branch on the row count.

Q. How do you generate thread ids?

A. f'leave-{id}-{uuid4().hex[:8]}' - readable prefix for debugging, random suffix so

re-triggering a leave gets a fresh thread instead of colliding with a completed one.

E. Data model & backend

Q. Why version timetable entries instead of updating them?

A. Every solve writes a new version, so history is preserved and you can diff what changed between runs. It also makes preserving-regeneration possible - we read the latest version back as the previous solution. And substitutions never mutate entries at all, which keeps 'what was ORIGINALLY scheduled' answerable forever.

Q. Walk me through the exchange tables.

A. period_exchanges holds absent_entry_id + leave_date on one side, partner_entry_id + recovery_date on the other, with both teacher ids, a plan_id, a status and a rationale string. A row with a NULL partner means 'no exchange available - needs manual arrangement', which is how lab blocks and genuinely stuck lessons surface honestly instead of silently vanishing.

Q. How does the dated view work if you never mutate the timetable?

A. GET /api/timetable/effective/{section}?date= takes the base weekday grid for that section, then overlays confirmed exchanges touching that date from either side - leave side or recovery side. COMPUTED, not stored, so an approval or rejection changes every future view instantly with no backfill.

Q. You mention an N+1 problem you fixed. What was it?

A. The adjacency guard ran per candidate per affected lesson, and each check hit the database - seconds per plan. I moved every read into a PlanContext built ONCE in about five queries with joinedload, then made every check a dict or set lookup. Identical logic, no queries inside the loop. Clearest performance lesson in the project: the algorithm wasn't slow, the I/O pattern was.

Q. Why create_all() + a hand-rolled ensure_schema() instead of Alembic?

A. A deliberate trade while the schema was still moving weekly - Alembic was deferred until it stabilised. ensure_schema() adds missing columns additively so an existing dev DB keeps working across phases; it cannot rename or drop, which is exactly why new concepts arrived as new TABLES rather than altered ones. For production this is the first thing I'd replace - and I'd say so without being pushed.

Q. Is the seed data idempotent?

A. Yes - seed() checks before inserting and seed_phase24() is purely additive, so

restarting the server never duplicates rows. It's what makes 'delete the DB and run' a reliable demo reset.

F. Security

Q. How does auth work?

A. JWT bearer tokens, HS256, with sub + role claims and an expiry. Passwords hashed with pbkdf2_sha256 via passlib - chosen over bcrypt because it's pure Python and needs no native build on Windows. get_current_user is a FastAPI dependency that decodes and loads the user; require_role('admin') is a dependency FACTORY applied per route.

Q. Can a student call the generate endpoint?

A. No - it carries dependencies=[Depends(require_role('admin'))] and returns 403. Reads are open to any authenticated user; every write is role-gated. We verified 200/403/401 across the setup router by curl.

Q. Can the LLM escalate privileges?

A. It has no route to. It emits ops; ops are applied by a request that ALREADY passed require_role('admin'). The model never holds a token, never chooses a route's permissions, never writes SQL. Authorisation is entirely outside the model's reach - which is the point of the typed-tools rule.

Q. What about prompt injection - a subject name containing instructions?

A. Real risk, partly mitigated. Entity names DO flow into the assistant's context. The mitigation is that the output surface is CLOSED: the worst a successful injection achieves is a different VALID op, which still has to pass validation and still gets rendered in English for the admin to approve. It can't reach SQL or the filesystem. I wouldn't call it solved - I'd add provenance markers around user-supplied data.

Q. Anything you know is insecure right now?

A. Yes, three things, all dev-appropriate and all one-liners to fix: CORS is allow_origins=['*'], the JWT secret has a placeholder default, and the token sits in localStorage rather than an httpOnly cookie, so it's XSS-reachable. Named, not hidden.

8. THE HARD ONES - ANSWER HONESTLY

These separate a candidate who BUILT something from one who can only demo it. Owning a limitation before it's found reads as engineering judgement; defending it reads as inexperience.

> Rule of thumb
> For every limitation: NAME IT -> say why the trade was right at the time -> say what you'd do instead.
>
> Three sentences. Then STOP TALKING.

!! Q. Where are your tests?

A. There is NO automated test suite - verification was manual and scripted: curl against every endpoint for 200/403/401, independent SQL clash checks against stored timetables, and both the approve and reject paths of the substitution flow run end-to-end and logged in the roadmap. That's real verification, but it isn't regression-proof, and it's the biggest gap in the project.

A. What I'd write first, in order: property tests on the solver (random inputs, assert no clashes + exact period counts on every returned solution), unit tests for each of the 23 op validators, and one integration test for interrupt -> approve -> resume including the double-execution idempotency.

!! Q. Is the booking agent real?

A. No - and I'd say so plainly. It does real room lookup from the DB and matches the venue from the query, but it does NOT write a Booking row or check conflicts against the timetable; its own response text says the confirmation is provisional. Same for the knowledge agent - an LLM answer with no retrieval; Chroma is deferred and commented out in requirements. Phase 3 is F3 booking + F4 RAG. The FINISHED features are the timetable and the leave/substitution flow, and those are what I'd demo.

!! Q. You claim CP-SAT is '175x faster' than genetic algorithms. Source?

A. That's a figure from the literature comparison in our research review, NOT a benchmark I ran on identical inputs. What I can defend from my own data is our measured numbers - ~8s cold, ~0.6s preserving, optimal with proof - and the qualitative claim that a GA cannot explain infeasibility, which is architectural, not a

matter of speed. I'd phrase the comparison as citation, not measurement.

Q. SQLite in production? Really?

A. No, and it isn't meant to be. It's the dev default; the DSN is configuration, the ORM is Postgres-compatible, and there is no SQLite-specific SQL in the codebase. Two things need attention on the move: the LangGraph checkpointer switches to the Postgres saver, and the approval race becomes a conditional update. Concurrent WRITERS are the actual limit, not size.

Q. What was the hardest bug you hit?

A. Two worth telling. (1) A plan took seconds to build and I assumed the adjacency algorithm was expensive - it wasn't, it was an N+1 query storm inside a nested loop. Preloading a context in five queries made it instant with the logic untouched.

A. (2) Subtler: regeneration returned a completely different, equally optimal timetable after a one-subject change, because CP-SAT has no notion of 'the answer you gave last time'. Hints alone didn't fix it - the fix was making stability an OBJECTIVE term.

Q. What would you do differently if you started again?

A. Three things. Alembic from day one instead of create_all + additive column patches. Tests alongside the solver rather than after - constraint code is exactly where property testing pays. And I'd have arrived at the constraint registry EARLIER: the first two phases hand-assembled solver options at the call site, and every rule change touched three files. Once the catalog existed, adding a rule kind became one registration.

Q. What's genuinely novel here vs assembled from libraries?

A. The libraries are the easy part. What's ours is the composition and four specific pieces:

- The period-exchange model with its dated overlay, which preserves the published timetable THROUGH a leave instead of rewriting it.
- Infeasibility explanation via named assumption-literal groups + cheap prechecks.
- Minimal-change regeneration as an OBJECTIVE term, not just a hint.
- The closed loop where a natural-language request becomes typed, validated, English-previewed operations against the SAME rulebook a human edits by hand.

Nobody hands you those in a package.

Q. What was YOUR contribution? (four-person team)

A. Have a specific, honest answer ready WITH FILE NAMES - solver/timetable_model.py, tools/exchange.py, whichever were mine. Being precise about my share, including what I didn't write, is more convincing than claiming all of it. Then offer to walk through the one file I know best, in depth.

9. THE 5-MINUTE DEMO PATH

Demo the two FINISHED features. Don't wander into booking or knowledge unless asked.

1. **Log in as admin, open /timetable**
   Show a clash-free grid - labs as blocks, an elective band as one violet cell listing every basket. Point out that the rules that produced it are listed, and link to /constraints.

2. **Type one sentence into the assistant**
   "Add a teacher for the new second-semester class." Show the ENGLISH PREVIEW of every op before anything is written - that preview IS the safety argument.

3. **Regenerate**
   Under a second, existing lessons unmoved. Say the sentence about hints vs the penalty term.

4. **Break it on purpose**
   Add an impossible rule, regenerate, show the NAMED conflicting constraint groups + suggested fixes. This is the moment that beats a genetic algorithm, and the panel will remember it.

5. **Apply a leave as faculty, approve as admin**
   Nobody prompts an agent. The approval card appears with the exchange plan, warnings and rationale per lesson. Approve, then show the dated grid on /exchanges and the notifications in both inboxes.

Last thing before you walk in

Lead with the ONE idea: the LLM orchestrates, the solver decides, a human approves.

Have the 4 novel pieces ready (p. see section 8).

Name the 3 gaps before they find them: no tests, booking + RAG are stubs, the 175x is a citation.

If a question goes somewhere you don't know - say so, then say how you'd find out. That answer scores better than a guess.

- Sujan